

Lecture 5:

Further enhancement to Scoring and results assembly and evaluation

Today's Lecture

- Speeding up vector space ranking
- Putting together a complete search system
 - Will require learning about a number of miscellaneous topics and heuristics
- Evaluation of the IR System

Efficient Cosine Ranking

- Find the K docs in the collection “nearest” to the query K largest query-doc cosines.
- Efficient ranking:
 - Computing a single cosine efficiently.
 - Choosing the K largest cosine values efficiently.
- Can we do this without computing all N cosines?

Computing Cosine Scores

COSINESCORE(q)

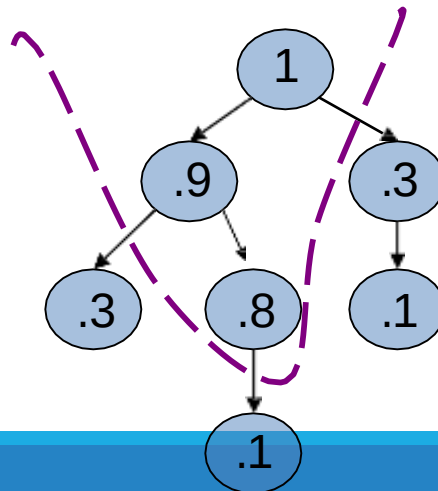
```
1  float Scores[ $N$ ] = 0
2  float Length[ $N$ ]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[ $d$ ] + =  $w_{t,d} \times w_{t,q}$ 
7  Read the array Length
8  for each  $d$ 
9  do Scores[ $d$ ] = Scores[ $d$ ] / Length[ $d$ ]
10 return Top  $K$  components of Scores[]
```

Computing the K largest Cosines: Selection vs. Sorting

- Typically we want to retrieve the top K docs (in the cosine ranking for the query)
 - not to totally order all docs in the collection
- Can we pick off docs with K highest cosines?
- Let J = number of docs with non zero cosines
 - We seek the K best of these from J

Use Heap for Selecting Top K

- Binary tree in which each node's value $>$ the values of children (max heap)
- Takes $2J$ operations to construct, then each of K “winners” read off in $2\log J$ steps.
- For $J=1\text{M}$, $K=100$, this is about 10% of the cost of sorting



Bottlenecks

- Primary computational bottleneck in scoring: cosine computation
- Can we avoid all this computation?
- Yes, but may sometimes get it wrong
 - a doc *not* in the top K may creep into the list of K output docs
 - Is this such a bad thing?

Cosine Similarity is only a Proxy

- User has a task and a query formulation
- Cosine matches docs to query
- Thus cosine is anyway a proxy for user happiness
- If we get a list of K docs “close” to the top K by cosine measure, should be ok

GenericApproach

- Find a set A of *contenders*, with $K < |A| \ll N$
 - A does not necessarily contain the top K , but has many docs from among the top K
 - Return the top K docs in A
- Think of A as pruning non-contenders
- The same approach is also used for other (non-cosine) scoring functions
- Will look at several schemes following this approach

Pruning Non-Contenders

- Index elimination
- Champion lists
- Static quality scores
- Impact ordering
- Cluster pruning

Index Elimination

- Basic cosine computation algorithm only considers docs containing at least one query term
- Take this further:
 - Only consider high-idf query terms
 - Only consider docs containing many query terms

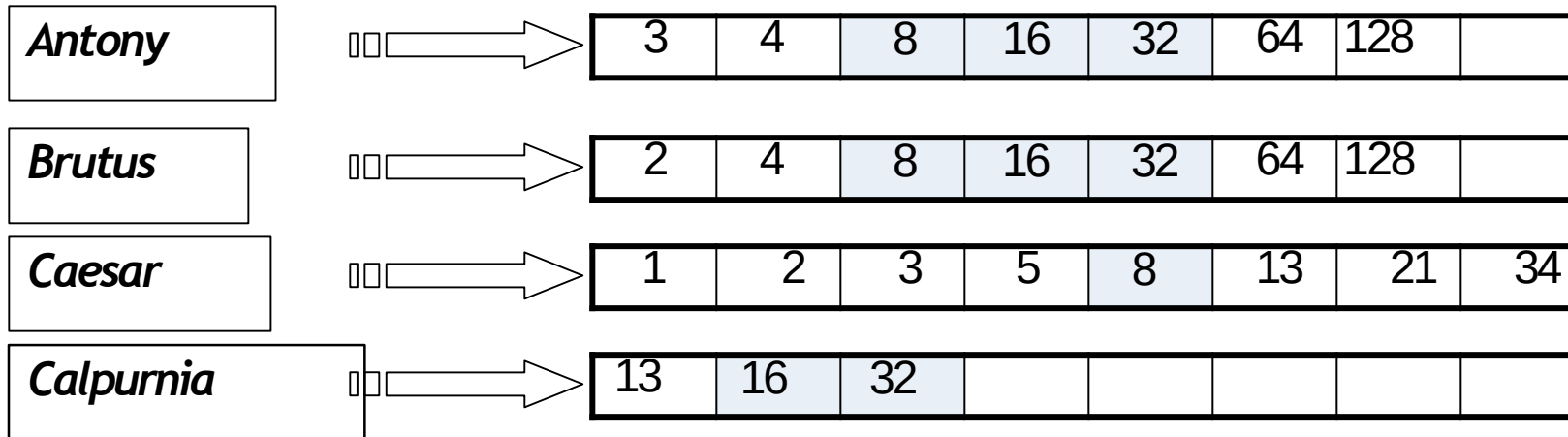
High-idf Query Terms Only

- For a query such as *catcher in the rye*
- *Only accumulate scores from catcher and rye*
- Intuition: *in* and *the* contribute little to the scores and so don't alter rank-ordering much
- Benefit:
 - *Postings of low-idf terms have many docs. these(many) docs get eliminated from set A of contenders*

Docs Containing Many Query Terms

- Any doc with at least one query term is a candidate for the top K output list
- For multi-term queries, only compute scores for docs containing several of the query terms
 - Say, at least 3 out of 4
 - Imposes a “soft conjunction” on queries seen on web search engines (early Google)
- Easy to implement in postings traversal

3 of 4 Query Terms



Scores only computed for docs 8, 16 and 32.

Champion lists

- Precompute for each dictionary term t , the r docs of highest weight in t 's postings
 - Call this the champion list for t
 - (aka fancy list or top docs for t)
- At query time, only compute scores for docs in the champion list of some query term
 - Pick the K top-scoring docs from amongst these
- Note that r has to be chosen at index build time
 - Thus, it's possible that $r < K$

Static quality scores

- We want top-ranking documents to be both *relevant* and *authoritative*
- *Relevance* is being modeled by cosine scores
- *Authority* is typically a query-independent property of a document
- Examples of authority signals
 - ❖ Wikipedia among websites
 - ❖ Articles in certain newspapers
 - ❖ A research paper with many citations
 - ❖ Many bitly's, diggs marks
 - ❖ (Pagerank)

Modeling authority

- Assign to each document a *query-independent* quality score in $[0,1]$ to each document d
 - Denote this by $g(d)$
- Thus, a quantity like the number of citations is scaled into $[0,1]$

Net score

- Consider a simple total score combining cosine relevance and authority
- $\text{net-score}(q,d) = g(d) + \text{cosine}(q,d)$
 - Can use some other linear combination
 - Indeed, any function of the two “signals” of user happiness – more later
- Now we seek the top K docs by net score

Top K by net score - fast methods

- First idea: Order all postings by $g(d)$
- Key: this is a common ordering for all postings
- Thus, can concurrently traverse query terms' postings for
 - Postings intersection
 - Cosine score computation

Why order postings by $g(d)$?

- Under $g(d)$ -ordering, top-scoring docs likely to appear early in postings traversal
- In time-bound applications (say, we have to return whatever search results we can in 50 ms), this allows us to stop postings traversal early
 - Short of computing scores for all docs in postings

Champion lists in $g(d)$ -ordering

- Can combine champion lists with $g(d)$ - ordering
- Maintain for each term a champion list of the r docs with highest $g(d) + \text{tf-idf}_{td}$
- Seek top- K results from only the docs in these champion lists

High and low lists

- For each term, we maintain two postings lists called *high* and *low*
 - Think of *high* as the champion list
- When traversing postings on a query, only traverse *high* lists first
 - If we get more than K docs, select the top K and stop
 - Else proceed to get docs from the *low* lists
- Can be used even for simple cosine scores, without global quality $g(d)$
- A means for segmenting index into two tiers
- Document-at-a-time scoring

Cluster pruning

Preprocessing:

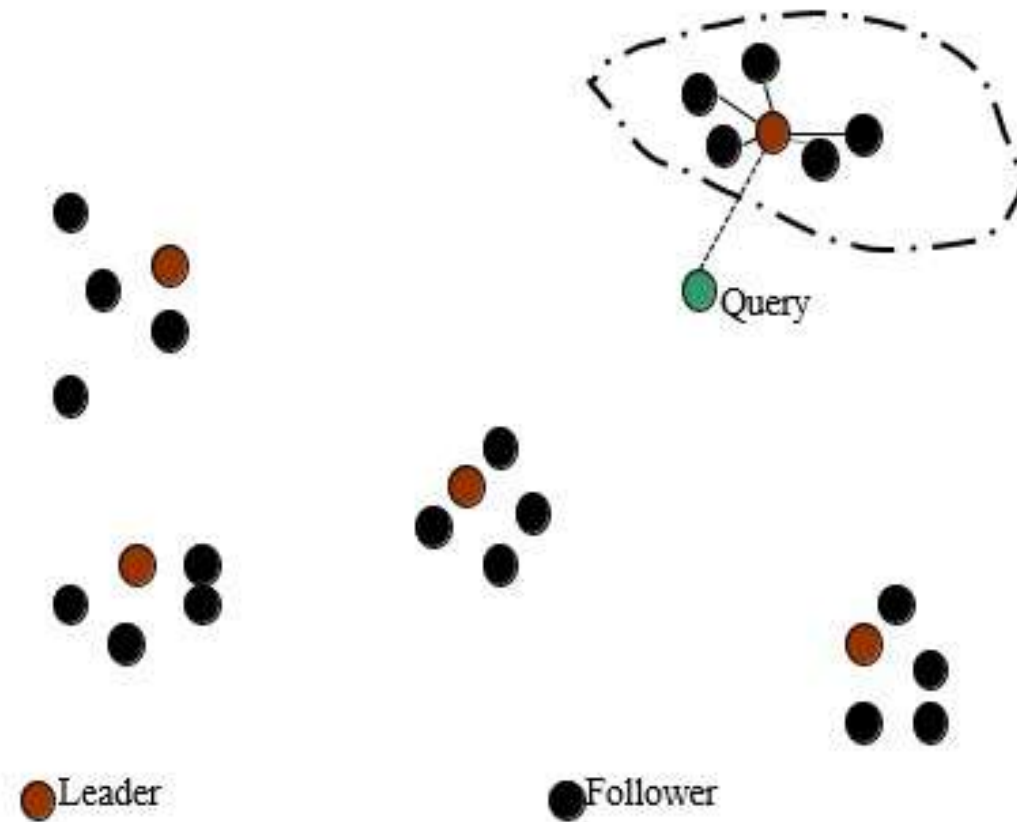
- Pick $\sqrt{N}$ docs at random: call these *leaders*
- For every other doc, pre-compute nearest leader
 - Docs attached to a leader: its *followers*;
 - Likely: each leader has $\sim \sqrt{N}$ followers.

Cluster pruning

Query Processing:

- Process a query as follows:
 - Given query Q , find its nearest *leader* L .
 - Seek K nearest docs from among L 's followers.

Visualization



Parametric and zone indexes

- Thus far, a doc has been a sequence of terms
- In fact documents have multiple parts, some with special semantics:
 - Author
 - Title
 - Date of publication
 - Language
 - Format
 - etc.
- These constitute the metadata about a document

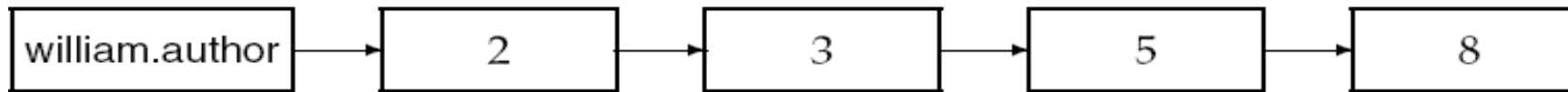
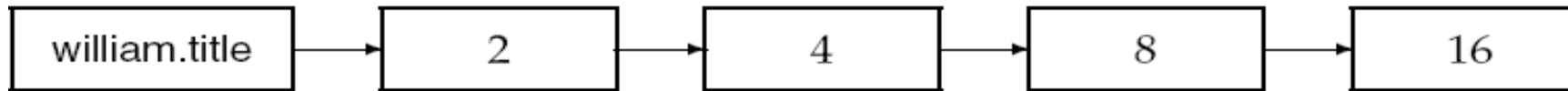
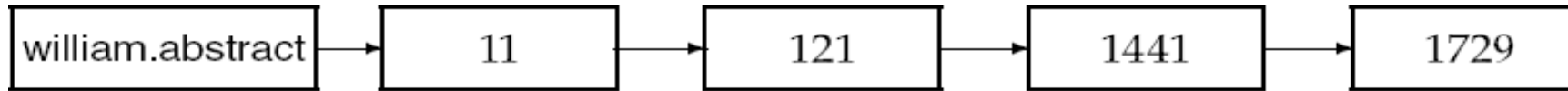
Fields

- We sometimes wish to search by these metadata
 - E.g., find docs authored by William Shakespeare in the year 1601, containing *alas poor Yorick*
- Year = 1601 is an example of a field
- Also, author last name = shakespeare, etc.
- Field or parametric index: postings for each field value
 - Sometimes build range trees (e.g., for dates)
- Field query typically treated as conjunction
 - (doc *must* be authored by shakespeare)

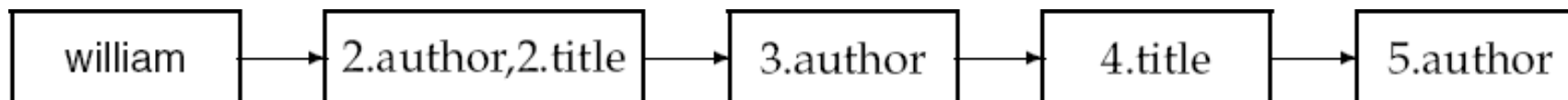
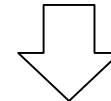
Zone

- A zone is a region of the doc that can contain an arbitrary amount of text, e.g.,
 - Title
 - Abstract
 - References ...
- Build inverted indexes on zones as well to permit querying
- E.g., “find docs with *merchant* in the title zone and matching the query *gentle rain*”

Example zone indexes



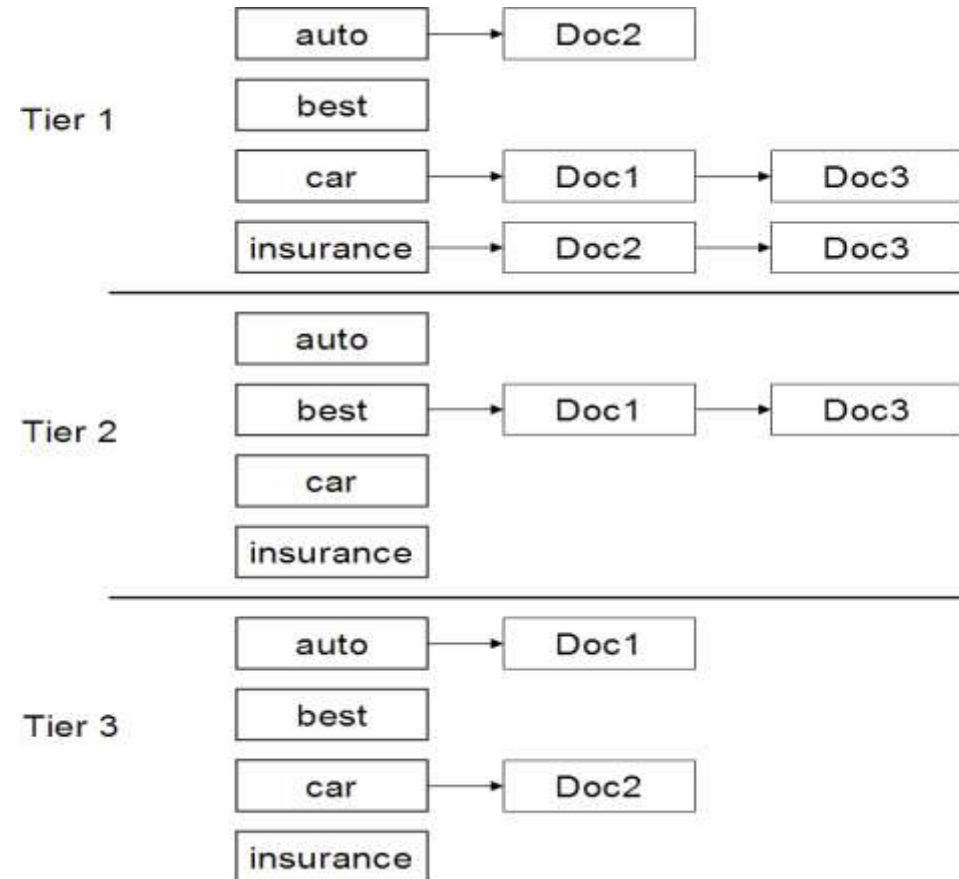
Encode zones in dictionary vs. postings.



Tiered indexes

- Break postings up into a hierarchy of lists
 - Most important
 - ...
 - Least important
- Can be done by $g(d)$ or another measure
- Inverted index thus broken up into tiers of decreasing importance
- At query time use top tier unless it fails to yield K docs
 - If so drop to lower tiers

Example tiered index



Query term proximity

- Free text queries: just a set of terms typed into the query box - common on the web
- Users prefer docs in which query terms occur within close proximity of each other
- Let w be the smallest window in a doc containing all query terms, e.g.,
 - For the query *strained mercy* the smallest window in the doc *The quality of mercy is not strained* is 7 (words)
- Would like scoring function to take this into account - how?

Impact-ordered postings

- We only want to compute scores for docs for which $wf_{t,d}$ (weighted term-frequency) is high enough
- Term-at-a-time scoring
- We sort each postings list by $wf_{t,d}$
- Now: not all postings in a common order!
- How do we compute scores in order to pick off top K ?
 - Two ideas follow

1. Early termination

- When traversing t 's postings, stop early after either
 - a fixed number of r docs
 - $wf_{t,d}$ drops below some threshold
- Take the union of the resulting sets of docs
 - One from the postings of each query term
- Compute only the scores for docs in this union

2.idf-ordered terms

- When considering the postings of query terms
- Look at them in order of decreasing idf
 - High idf terms likely to contribute most to score
- As we update score contribution from each query term
 - Stop if doc scores relatively unchanged
- Can apply to cosine or some other net scores

Query parsers

- Free text query from user may in fact spawn one or more queries to the indexes, e.g., query *rising interest rates*
 - Run the query as a phrase query
 - If $< K$ docs contain the phrase *rising interest rates*, run the two phrase queries
 - If we still have $< K$ docs, run the vector space query

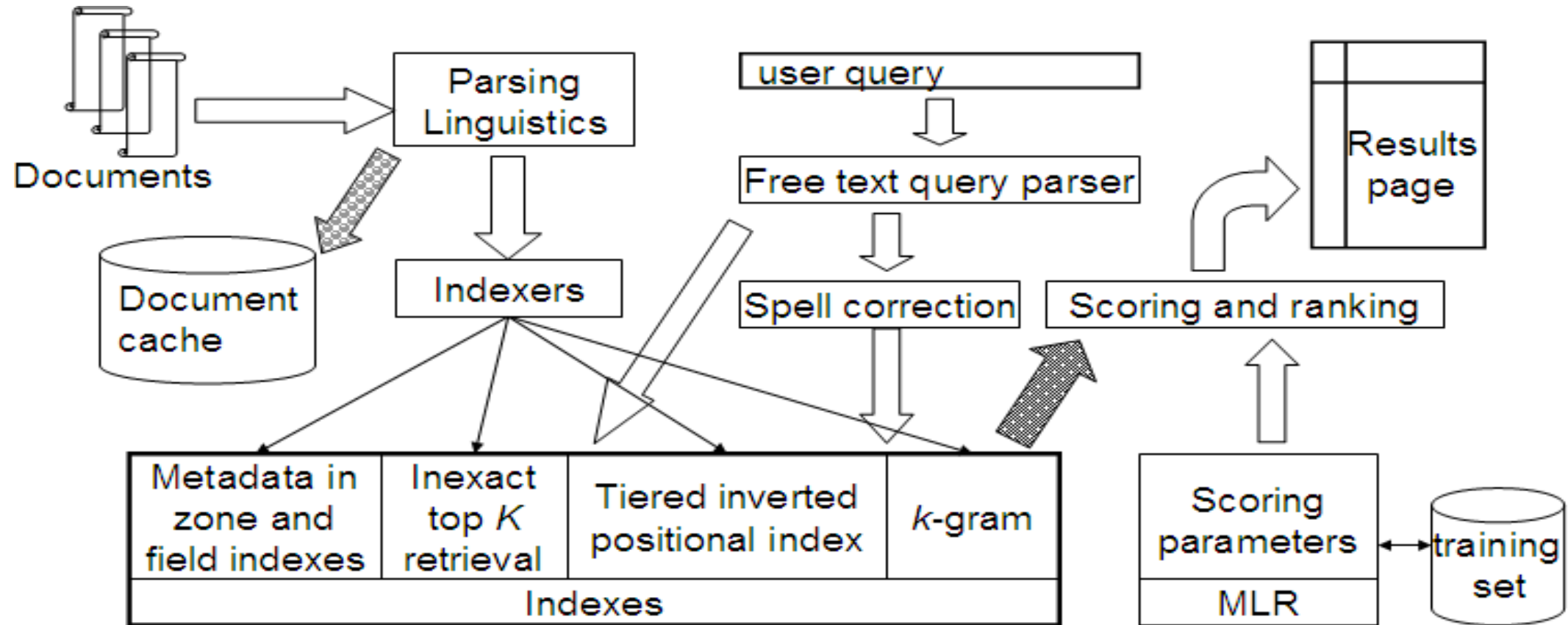
rising interest rates

 - Rank matching docs by vector space scoring
- This sequence is issued by a query parser

Aggregate scores

- We've seen that score functions can combine cosine, static quality, proximity, etc.
- How do we know the best combination?
- Some applications - expert-tuned
- Increasingly common: machine-learned

Putting it all together



Evaluation of Information Retrieval Systems

- How do we evaluate??
 - Try a few queries on his engine and say “Not bad”?
 - Is it enough??
- How fast does it index?
 - Number of documents/hour
 - Incremental indexing
- How fast does it search?
 - Latency and CPU needs
- Does it recommend related products/links?

Measure for a search engine

- All of the preceding criteria are measurable
 - We can make expressiveness precise
- The key measurement: User happiness
 - What is this?
 - Speed of response/size of index are factors
 - But blindingly fast, useless answers won't make a user happy
 - Need a way of quantifying user happiness

Measuring user happiness

- Issue :who is the user we are trying to make happy?
 - Depends on setting
- Web Engine
 - User finds what he/she wants and returns to the engine
- eCommerce site

Happiness: Elusive to measure

- Most common proxy: relevance of search results
- But how do you measure relevance?
- Three elements:
 1. A benchmark document collection
 - CRANFIELD, REUTERS ,TREC
 2. A benchmark suite of queries
 3. An assessment of either Relevant or Nonrelevant for each query and each document
 - Relevance is assessed relative to an information need, not a query

Evaluating an IR system

- Note : User need is translated into a query
- Relevance is assessed relative to the user need, not the query
- E.g, Information need: My swimming pool bottom is becoming black and needs to be cleaned
- Query: Pool cleaner
- Assess whether the doc addresses the underlying need, not whether it has these words

Binary Assessments

Precision: fraction of retrieved docs that are relevant =
 $P(\text{relevant} | \text{retrieved})$

Recall: fraction of relevant docs that are retrieved
 $= P(\text{retrieved} | \text{relevant})$

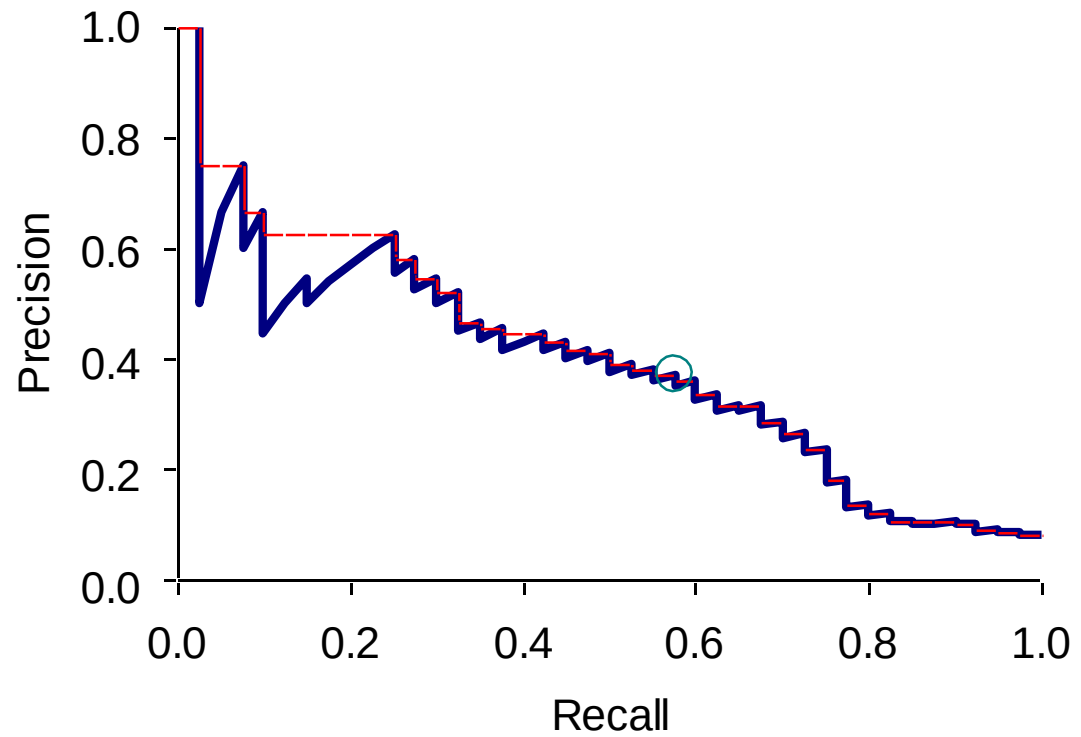
	Relevant	Nonrelevant
Retrieved	tp	fp
Not Retrieved	fn	tn

- Precision $P = tp / (tp + fp)$
- Recall $R = tp / (tp + fn)$

F Measure: A single measure that trades off precision versus recall

$$F_1 = \left(\frac{2}{\text{recall}^{-1} + \text{precision}^{-1}} \right) = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Precision-recall Curve



Rank Based Measures

Binary relevance

- Precision@K(P@K)
- Mean Average Precision (MAP)

Precision@K(P@K)

- Set a rank threshold K
- Compute % relevant in top K
- Ignores documents ranked lower than K

Ex:

- Prec@3 of 2/3
- Prec@4 of 2/4
- Prec@5 of 3/5



Mean Average Precision

- Consider rank position of each relevant doc ($K_1, K_2, \dots, K_r$)
- Compute Precision@K for each $K_1, K_2, \dots, K_r$
- Average Precision = average of P@K
- MAP is average Precision across multiple queries/rankings

Average Precision

 = the relevant documents

Ranking #1

										
Recall	0.17	0.17	0.33	0.5	0.67	0.83	0.83	0.83	0.83	1.0
Precision	1.0	0.5	0.67	0.75	0.8	0.83	0.71	0.63	0.56	0.6

Ranking #2

										
Recall	0.0	0.17	0.17	0.17	0.33	0.5	0.67	0.67	0.83	1.0
Precision	0.0	0.5	0.33	0.25	0.4	0.5	0.57	0.5	0.56	0.6

Ranking #1: $(1.0 + 0.67 + 0.75 + 0.8 + 0.83 + 0.6)/6 = 0.78$

Ranking #2: $(0.5 + 0.4 + 0.5 + 0.57 + 0.56 + 0.6)/6 = 0.52$

Mean Average Precision

 = relevant documents for query 1

Ranking #1

										
Recall	0.2	0.2	0.4	0.4	0.4	0.6	0.6	0.6	0.8	1.0
Precision	1.0	0.5	0.67	0.5	0.4	0.5	0.43	0.38	0.44	0.5

 = relevant documents for query 2

Ranking #2

										
Recall	0.0	0.33	0.33	0.33	0.67	0.67	1.0	1.0	1.0	1.0
Precision	0.0	0.5	0.33	0.25	0.4	0.33	0.43	0.38	0.33	0.3

Mean Average Precision

average precision query 1 = $(1.0 + 0.67 + 0.5 + 0.44 + 0.5)/5 = 0.62$

average precision query 2 = $(0.5 + 0.4 + 0.43)/3 = 0.44$

mean average precision = $(0.62 + 0.44)/2 = 0.53$